Ontwikkeling van een visuele tijdsbeheertool voor Xplanner met re-frame.

Iris Van Damme.

Graduaat in het Programmeren

Vak: Graduaatsproef
Stagebedrijf: Lorre Engineering
Academiejaar: 2025–2026
Eerste examenperiode

Departement IT en Digitale Innovatie .

HO GENT

Woord vooraf

Dit verslag bespreekt het programmeren van een front-end voor een dynamische webapplicatie met behulp van re-frame, een framework voor de data-driven functionele programmeertaal Clojure. Mijn stage heb ik gevolgd bij Lorre Engineering. Voor de graduaatsproef kregen we de opdracht om gedurende 10 dagen op de stageplek individueel een software-project te volbrengen. Dit verslag omvat wat ik tijdens deze 10 dagen uitgewerkt heb.

Graag wil ik de medewerkers van Lorre Engineering bedanken voor mijn het creëren van een goede, gefocuste werkomgeving. Tenslotte wil ik de docenten van Hogeschool Gent bedanken voor de goede begeleiding tijdens dit traject.

Samenvatting

Lorre Engineering, mijn stagebedrijf tijdens dit eerste semester, voert zijn Scrumplanning- en registratie uit in een programma genaamd Xplanner. Hiervoor ontwikkelde ik een nieuwe visuele gebruikersinterface omdat de originele interface niet de juiste functionaliteiten bezit. Zo wordt er met dit project een visuele kalenderweergave "timesheet" gecreëerd waarin de softwareontwikkelaars en manager alle taken overzichtelijk kunnen bekijken, en nieuwe "time entries" kunnen creëren en bewerken. De nieuwe technologie die ik hiervoor aanleerde is re-frame, een framework om React binnen ClojureScript (compiler die Clojure omzet in JavaScript) te integreren 8 (2021). De nieuwe tijdstool krijgt de data uit de originele Xplanner back-end via een SOAP-API. Zo bleef de implementatie voor de graduaatsproef voornamelijk beperkt tot een front-end. Ten eerste heb ik toch een klein back-end Clojure-project gecreëerd dat de serveromgeving start en de SOAP-API-calls uitvoert. Deze stuurt vervolgens REST-API-requests naar het front-end re-frame-project dat de gebuikersinterface-code bevat.

Inhoudsopgave

Lijst van figuren vi

Lijst van codefragmenten vii

1 Inleiding 1
- 1.1 Probleemstelling . . . 1
- 1.2 Onderzoeksvraag . . . 1
- 1.3 Onderzoeksdoelstelling . . . 2
- 1.4 Opzet van deze graduaatsproef . . . 2

2 Nieuwe Technologieën 3
- 2.1 Functioneel programmeren . . . 3
- 2.2 Clojure . . . 3
- 2.3 ClojureScript en Reagent . . . 4
- 2.4 re-frame . . . 4

3 Structuur applicatie en motivatie 5
- 3.1 Clojure back-end project . . . 5
- 3.2 re-frame front-end project . . . 5

4 Resultaat 8

5 Evaluatie van het ontwikkelingsproces 11
- 5.1 Projectaanpak . . . 11
- 5.2 Clojure . . . 11
- 5.3 SOAP . . . 12
- 5.4 re-frame . . . 12

6 Conclusie 13

Bibliografie 14

Lijst van figuren

3.1 Structuur van de applicatie 6

4.1 Resultaat: “timesheet” gebruikersinterface 9
4.2 Resultaat: Wijzig “time entry” 10

Lijst van codefragmenten

2.1 Voorbeeld van een re-frame event (pure function). 4
2.2 Voorbeeld van een re-frame subscription (pure function). 4

1

Inleiding

Bij Lorre Engineering gebruikt men een programma genaamd Xplanner om de Scrumplanning en tijdsregistratie in te vullen. De bestaande gebruikersinterface biedt niet genoeg mogelijkheden om de taken binnen de beschikbare tijd te visualiseren. Voor mijn graduaatsproef is er op deze manier gekozen voor de ontwikkeling van een dynamische webapplicatie met nieuwe gebruikersinterface voor Xplanner. Tijdens mijn stage heb ik al enkele statische webpagina's geprogrammeerd, maar aangezien er een stabiel framework bestaat om dynamische webpagina's te coderen was het voor het bedrijf en mezelf een meerwaarde om te onderzoeken of dit een efficiëntere aanpak kan vormen.

1.1. Probleemstelling

Binnen het bedrijf Lorre Engineering wordt de Scrumplanning- en registratie ingevuld in Xplanner. De gespendeerde tijden, "time entries", worden getoond in tekstformaat met een begin- en eindtijd. Wanneer er onverantwoorde tijd binnen de bedrijfsuren verstreken is omdat een programmeur zijn tijdsregistratie op dat moment vergeten is in te vullen, is het moeilijk na te gaan waar een leegte zit, want bedrijfsmanager en applicatie-ontwikkelaar kunnen visueel niet zien hoe hun tijdsbesteding eruit ziet. Bovendien moeten de bestede uren steeds achteraf manueel ingetikt worden en is er geen functionaliteit die een taak in realtime kan starten en stoppen.

1.2. Onderzoeksvraag

Kan er efficiënt een dynamische front-end applicatie ontwikkeld worden in re-frame om de manager en programmeurs een visueel overzicht te geven van de Scum taken tegenover de beschikbare tijd inclusief de functionaliteit om "time entries" te bewerken en nieuwe taken toe te voegen?

1.3. Onderzoeksdoelstelling

De Scrumplanning- en tijdsregistratie binnen Lorre Engineering vereenvoudigen en op een meer overzichtelijke manier weergeven door een nieuwe visuele interface te ontwikkelen.

1.4. Opzet van deze graduaatsproef

De rest van deze graduaatsproef is als volgt opgebouwd:
In Hoofdstuk 2 worden de nieuwe technologieën Clojure, ClojureScript, reagent en re-frame uitgelegd met initiële motivatie voor deze keuzes.
In Hoofdstuk 3 wordt de structuur van de applicatie uitgelegd.
In Hoofdstuk 4 wordt het resultaat weergeven.
In Hoofdstuk 5 is de finale ervaring met de technologieën en een evaluatie van de aanpak uitgeschreven
In Hoofdstuk 6, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvraag.

2

Nieuwe Technologieën

2.1. Functioneel programmeren

De graduaatsproef werd volledig uitgevoerd in Clojure, een functionele programmeertaal. Bij functioneel programmeren, in tegenstelling tot imperatief programmeren, wordt een serie van functies geschreven die met dezelfde input, gelijk welke, steeds dezelfde output zouden moeten bekomen. Dit soort functies heten pure functions. Microsoft (2021). Daarentegen wordt er bij imperatief programmeren van een bekende input uitgegaan waarop één voor één een actie uitgevoerd wordt. Neveneffecten worden bij functioneel programmeren als argument aan de functie meegegeven om de functie pure te behouden. Bij imperatief programmeren worden deze in de functie zelf opgevraagd, wat code onvoorspelbaar kan maken en het moeilijker maakt om te debuggen en te testen. Bij het functioneel programmeren maakt men dus geen gebruik van klassen en objecten, maar wordt de programmatie op een data-georiënteerde manier uitgevoerd.

2.2. Clojure

Clojure is een programmeertaal die de ontwikkelaar forceert om functioneel te programmeren via immutable variabelen, ontwikkeld door Rich Hickey met ondersteuning van vele Java-functionaliteiten. Het draait bijvoorbeeld op de JVM (Java Virtual Machine) en maakt gebruik van de Java garbage collection. Het is een veelzijdige programmeertaal, wordt zowel voor front-end als back-end gebruikt en in allerlei domeinen (ontwikkelen van webapplicaties, financiële applicaties, administratie, ...). De ontwikkeling van een Clojure-applicatie wordt als REPL-driven gerealiseerd, wat betekent dat de programmeur kleine afzonderlijke codesecties manueel kan compileren en runnen zonder dat het geheel gecompileerd moet worden. De aanbevolen aanpak is dan ook om kleine stukken code te testen alvorens deze in de applicatie te zetten. Er werd voor Clojure gekozen omdat het stagebedrijf

met gevoelige klantengegevens en asynchronische lees- en schrijfoperaties werkt. Wanneer verschillende threads dezelfde gegevens opvragen mag er zich in dit geval absoluut geen read/write-mismatch voordoen.

2.3. ClojureScript en Reagent

ClojureScript is een compiler die Clojure-code compileert naar JavaScript code, voornamelijk gebruikt bij het ontwikkelen van front-end webapplicaties. Reagent is een minimale interface om React te schrijven in een Clojure-project in combinatie met de Hiccup library voor de plaatsing van HTML tags. Helaas zijn React- en JavaScip-tevents en -subscriptions geen pure functions, ook niet in Reagent, dus hiervoor wordt de programmeur geholpen door de re-frame functionaliteiten.

2.4. re-frame

re-frame lost het probleem van de impure events en subscriptions in Reagent op. Indien er data in de lokale database (equivalent van React Redux-store) wordt opgeslagen wordt de huidige database samen met andere co-effecten aan het event als argumenten meegegeven. Een equivalente werking geldt voor subscriptions.

```
(rf/reg-event-db
:time-entries/delete-time-entry-success
(fn [db [_ time-entry]]
    (-> db
        (update-in
            [:projects (->kw (:project-id time-entry))
            :iterations (->kw (:iteration-id time-entry))
            :user-stories (->kw (:user-story-id time-entry))
            :tasks (->kw (:taskId time-entry))
            :time-entries] dissoc (->kw (:id time-entry))))))
```

Codefragment 2.1: Voorbeeld van een re-frame event (pure function).

```
(rf/reg-sub
:time-entries/time-entries
(fn [db [_ project-id iteration-id user-story-id task-id]]
    (get-in db [:projects (->kw project-id) :iterations (->kw
        iteration-id)
        :user-stories (->kw user-story-id) :tasks (->kw task-id)
            :time-entries])))
```

Codefragment 2.2: Voorbeeld van een re-frame subscription (pure function).

3

Structuur applicatie en motivatie

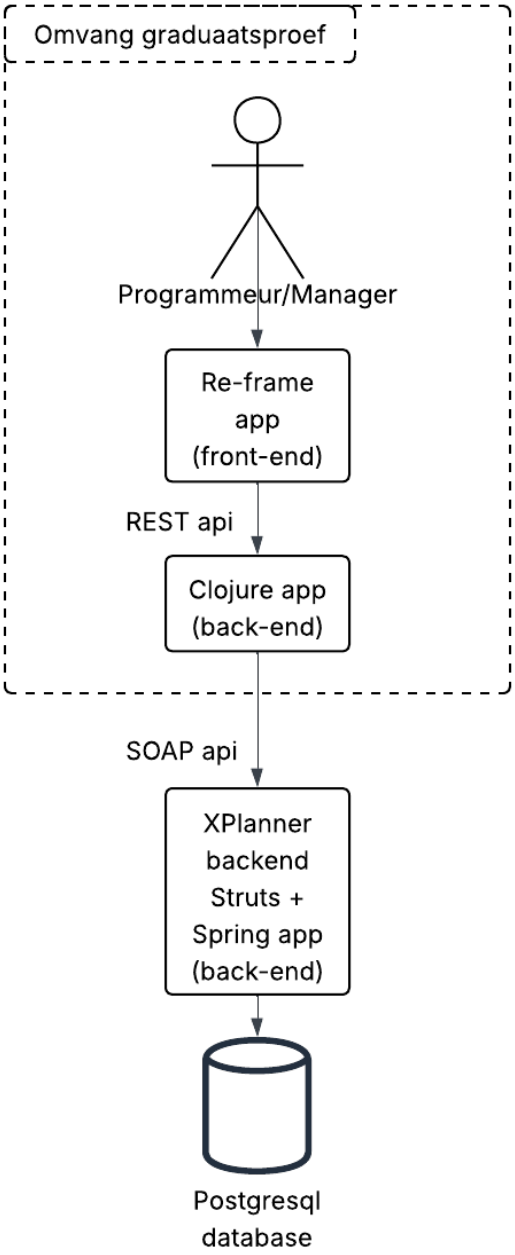
In dit hoofdstuk wordt er dieper ingegaan over de structuur van de applicatie. Indien relevant worden kleinere gebruikte technologieën nog kort uitgelegd. Dit hoofdstuk bespreekt tenslotte de initiële motivatie voor de structuur en aanpak.

3.1. Clojure back-end project

De back-end van de graduaatsproef is geschreven in Clojure. Aan de basis ligt Java Jetty server die een webserver en servletcontainers bouwt Eclipse Foundation (2026) omdat ik tijdens de stage ook gebruik maakte van Jetty-server en deze was steeds performant en "reliable". Vervolgens werd de Ring-library toegevoegd, die een abstractielaag creëert voor de programmatie van HTTP-serverapplicaties McGranaghan et al. (2025). Deze library benutte ik eveneens tijdens de stage en kon hiermee gemakkelijk en overzichtelijk request maps coderen, en handlers, middleware en routes configureren.
In deze Clojure-back-end werd ook de logincode geïmplementeerd. Hier wordt gebruik gemaakt van JWT tokens die opgeslagen worden als cookies. Er werd gekozen voor JWT tokens omdat deze geëncrypteerde gebruikersinformatie kunnen opslaan in de cookies zoals de gebruikersnaam en aan de serverkant moeten geen cookies opgeslagen worden in de database.
Deze Clojure-back-end vraagt de data op die het front-end nodig heeft vanuit de originele Xplanner-back-end door middel van een SOAP-API. In SOAP wordt data tussen applicaties verstuurd in XML-formaat. De request-parameters zijn geëncapsuleerd in een enveloppe.

3.2. re-frame front-end project

In de applicaties zijn pagina's door het front-end gecodeerd in re-frame en Hiccup. Om een dynamische webapplicatie te ontwikkelen worden net zoals in standaard



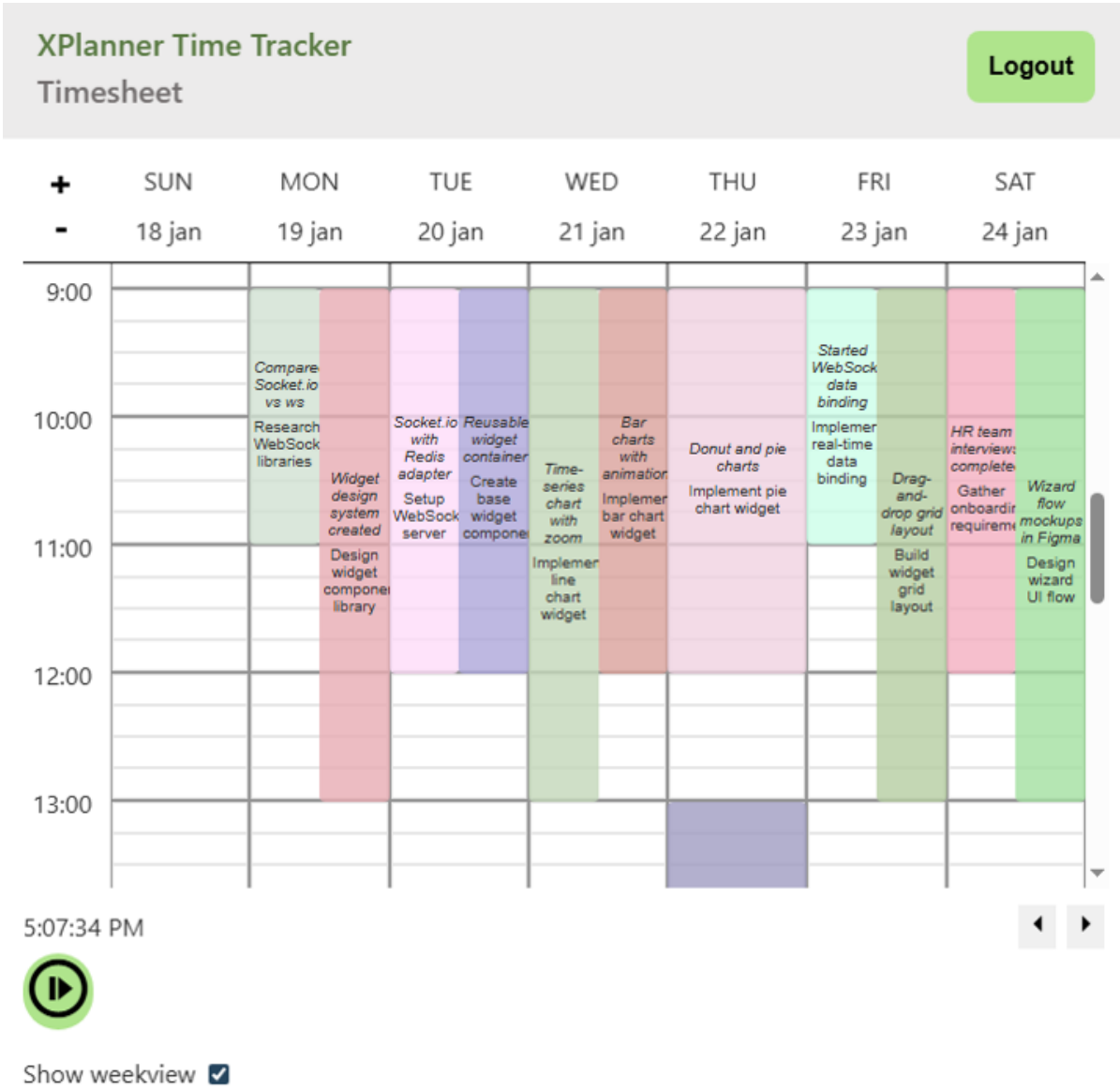
Figuur 3.1: Structuur van de applicatie.

React componenten genest om de pagina te renderen. De React-events sturen REST-API-requests naar de Clojure-back-end en slaan tijdelijke data (data van taken, time entries, ...) op in de lokale database (equivalent van React redux-store) nodig voor de gebruikersinterface. Om de data uit de lokaledatabase op te halen en in de gebruikersinterface te renderen, gebruiken we subscriptions.

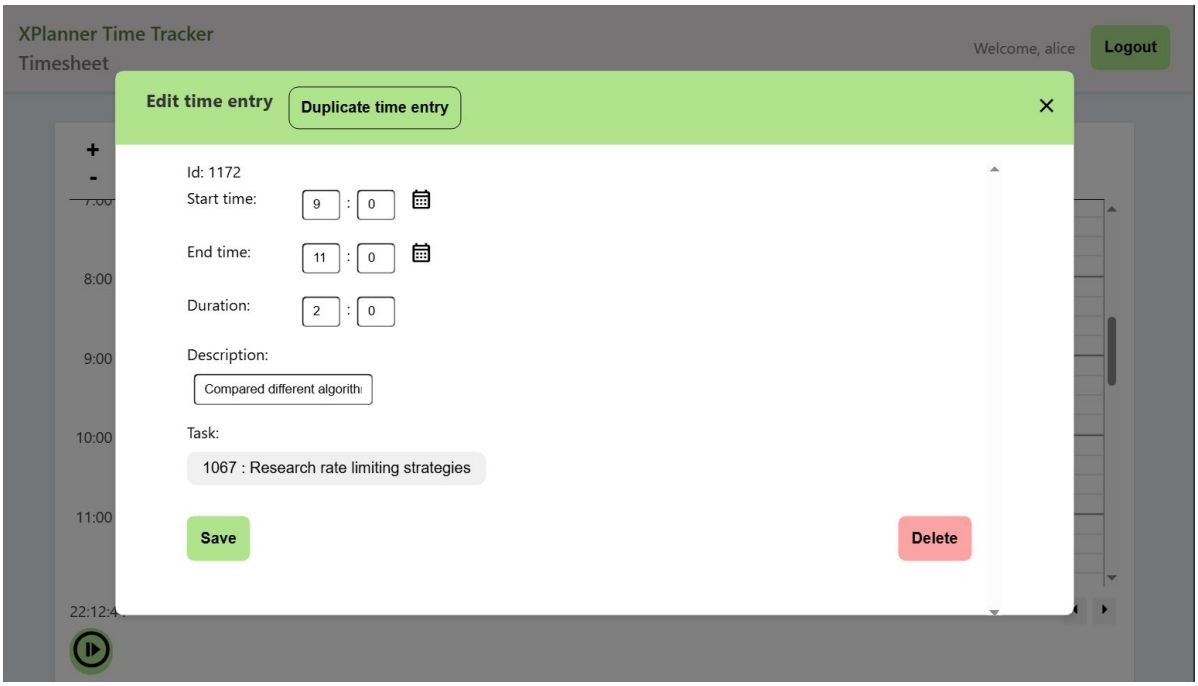
4

Resultaat

Het resultaat van deze graduaatsproef is een (voornamelijk) front-end re-frame-applicatie die in een visuele kalendervoorstelling de programmeur en manager inzicht geeft op de scrum taken tegenover de beschikbare tijd. Bovendien biedt het programma een interface om de “time entries” gemakkelijk te bewerken en nieuwe taken toe te voegen.
Vervolgens biedt de nieuwe applicatie de mogelijkheid om taken in realtime te starten en te stoppen. De nieuwe user interface is interactiever. Zo kan de gebruiker time entries” creëren door te verslepen (“draggen”) met de muis, en deze op dezelfde manier verkorten, verlengen, dupliceren en verplaatsen in de tijd.



Figuur 4.1: Resultaat: “timesheet” gebruikersinterface



Figuur 4.2: Resultaat: Wijzig “time entry”

5

Evaluatie van het ontwikkelingsproces

5.1. Projectaanpak

Ongeveer een maand op voorhand hebben we in team besproken wat de functionaliteiten de nieuwe gebruikersinterface zou moeten bevatten. Ik heb dan mockups ontworpen. Volgens een Agile aanpak hebben de verschillende wijzigingen die volgden het plan voor dit project bijgestuurd. Een ontwikkelingsproces van tien dagen is zeer kort. Met de programmeurs van het stagebedrijf hebben we na een paar dagen in de implementatie toch een MVP (Most Viable Product) opgesteld met de functionaliteiten die zeker in de applicatie aanwezig moesten zijn. Vervolgens om de dag in een meeting besproken hoe ver de vooruitgang stond.

5.2. Clojure

Clojure was de meest gebruikte programmeertaal tijdens de stage en mijn ervaring hiermee was vrij positief. Na enkele weken training is de syntax intuïtief en de mogelijkheid om vooraf codesegmenten in de REPL te testen zorgt later voor een kortere debugfase. Het vooruitzicht van het stagebedrijf is om in de toekomst nog meer testing te implementeren. Clojure leent zich hier goed voor. Helaas was een ontwikkelingstraject van 10 dagen was iets te kort om testen te implementeren voor deze applicatie. Door de pure functions in Clojure kunnen er echter gemakkelijk testen geschreven worden met de clojure.test-library voor unit testing of de test.check-library voor property-based testing.

5.3. SOAP

De SOAP-API-interface werkte naar behoren, maar is omslachtiger dan een REST API. Wanneer er enkel gebruik gemaakt wordt van het HTTP protocol zou ik een REST api verkiezen.

5.4. re-frame

Re-frame biedt alle mogelijkheden van React, maar ook de voordelen die Clojure met zich meebrengt. Ik kan dit framework zeker aanraden. Met voorafgaande kennis van React, Clojure en Hiccup (Clojure HTML-library) is er enkel een leercurve om de JavaScript-events en subscriptions in re-frame om te vormen naar pure functions, maar eens hierin goed getraind bieden deze meer voorspelbaarheid en zijn gemakkelijker te debuggen en te testen.
Voor de ontwikkeling van de applicatie werd de Fighweel-buildtool met hot reload gebruikt. Mijn ervaring was dat het "reliable" is, maar de REPL-compilatie traag kan zijn. Voor re-frame projecten in de toekomst zou ik Shadow-cljs eens willen testen Heller en Kay (2018). Een laatste minpunt zijn de foutmeldingen of "error messages". Zo kreeg ik vaak de error "Invalid hiccup code" zonder duidelijk regelnummer.

6

Conclusie

Een dynamische website ontwikkelen in een functionele programmeertaal lukt vlot en efficiënt met re-frame. Er is een grote leercurve, maar met voorkennis van Clojure en React lukt het om in korte tijd een schaalbare applicatie te ontwikkelen. Deze applicatie toont een visuele kalenderweergave voor de manager en programmeurs van Lorre Engineering om een overzicht te krijgen van de “time entries” van de taken in de beschikbare tijd. Via een gebruiksvriendelijke interface kunnen nieuwe “time entries” en taken aangemaakt en bewerkt worden.

Bibliografie

8, D. (2021). *re-frame Docs*. Geraadpleegd op 17 januari 2026, van https://day8.github.io/re-frame/re-frame/

Eclipse Foundation, I. (2026). *Jetty officiële website*. Geraadpleegd op 18 januari 2026, van https://jetty.org/

Heller, T., & Kay, T. (2018). *Shadow-cljs documentation*. Geraadpleegd op 18 januari 2026, van https://shadow-cljs.github.io/docs/UsersGuide.html

McGranaghan, M., Reeves, J., & contributors. (2025). *Ring Github*. Geraadpleegd op 18 januari 2026, van https://github.com/ring-clojure/ring/blob/master/SPEC.md

Microsoft. (2021). *Functional programming vs. imperative programming (LINQ to XML)*. Geraadpleegd op 17 januari 2026, van https://learn.microsoft.com/en-us/dotnet/standard/linq/functional-vs-imperative-programming